

Projet Contrôle et Apprentissage

Note: Pour essayer de changer un peu, cela nous intéressait de proposer un sujet sur la navigation de drone car l'exemple que vous avez donné au début du cours nous paraissait intéressant (nous sommes en ASR). En se renseignant un peu, on a vu que l'on pouvait aussi voir des méthodes algorithmiques intéressantes dans ce contexte pour faire fonctionner le Deep Learning avec du RL.

Cependant, comme on sait que normalement le thème du projet doit plutôt être sur du réseau, on propose ci-dessous un deuxième sujet sur de la cybersécurité. C'est pour cela que nous rendons ce fichier avec donc deux propositions de sujets. Il y a des points communs entre les deux notamment pour la partie implémentation.

Groupe

#1 Lilian CARPENTIER

#2 Louan GIROUD

#3 Arthur MORAIN

#4 Rémi AIRIAU

I. Première proposition

1. Titre du projet: Navigation autonome d'un drone dans un environnement contraint avec du Deep Q-Learning (DQL)

2. Problématique: La combinaison de Deep Learning avec l'Apprentissage par Renforcement nous intéressait fortement (notamment pour faire de l'embarqué par exemple). La problématique serait de faire naviguer un agent (drone) d'un point A à un point B sans intervention humaine, en évitant des obstacles et en gérant des forces physiques simples comme la gravité ou l'inertie. On doit transformer des entrées sensorielles continues en commandes moteurs optimales (en appliquant les algos de RL).

Composants du système:

- On a l'agent (modèle DQN) qui décide des actions à prendre en fonction des données qu'ils reçoit.
- L'environnement: c'est simplement un espace 2D avec des obstacles.
- Les capteurs simulés: on suppose que l'on connaît à tout instant la position, vitesse et l'angle du drone.
- Les moteurs du drone qui appliquent le mouvement.

Performance et objectifs: L'objectif est triple:

- Maximiser le taux de réussite (atteindre la cible soit le point B en partant de A).
- Minimiser le temps de vol. C'est une pénalité temporelle, on doit pouvoir atteindre la cible en un temps minimal.
- Assurer la sécurité. On doit éviter les collisions avec les obstacles (on pourrait même le faire avec un autre drone...), et peut-être aussi éviter les angles critiques (plus haut risque).

3. Définition du MDP

- Etats (S): Vecteur de dimension 6 = [X, Y, V_x, V_y, Angle d'inclinaison, Vitesse angulaire]

- Actions (A): 4 actions possibles. Si on a deux moteurs sur le drone (puisqu'on est en 2D), on a aucun moteur activé (L:0, R:0), une poussée à gauche avec inclinaison (L:1, R:0), une poussée à droite avec inclinaison (L:0, R:1) et une poussée complète (L:1, R:1). Si le drone est à plat, il monte verticalement. Sinon, il accélère dans sa direction actuelle.
- Transitions (fonction continue P): elles donnent la position après le choix de l'action a dans l'état s . On définit juste simplement les équations physiques de base: accélération, vitesse et position.
- Récompense: Par exemple +100 si la cible est atteinte, -100 si le drone s'écrase et -0.1 par unité de temps.
- Paramètres: pour l'exploration, on peut prendre $\epsilon = 0.1$ ou bien commencer plus haut comme 0.5 et le diminuer à chaque pas de temps jusqu'à 0.01 ou quelque chose comme ça pour fixer nos choix au fur et à mesure que le temps passe. Pour γ , on prend 0.99 pour pas que le drone aille très vite dans une direction et se crash juste après.

4. Implémentation

Pour la simulation, on va le faire en ligne de commande (dans la console), et/ou exporter des données par exemple en csv pour le visualiser avec des graphes en python.

On a voulu faire du Deep Q-Learning. De ce qu'on a compris, le Q-Learning permet de se passer de la connaissance de l'environnement et converger vers la solution optimale. On a déjà fait du Deep Learning donc c'est intéressant de lier les deux pour passer du discret au continu.

Détails algorithmiques:

On va utiliser la MSE comme Loss entre l'équation de Bellman $r + \gamma \max_{a'} Q(s', a')$ et la prédiction actuelle $Q(s, a)$. On va avoir quelque chose comme

$$L = [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]^2.$$

Experience Replay

On a vu sur internet que pour de la navigation, un simple réseau de neurones ne suffit pas.

Il faut implémenter de l'*Experience Replay*. Le principe de base est que d'un pas de temps à l'autre, les données sont très similaires donc le réseau va faire de l'overfitting sur sa position actuelle et prendre les mauvaises décisions.

On doit pouvoir le prouver mathématiquement puisqu'on connaît l'équation pour mettre à jour les Q-values qui dépend uniquement de l'état courant (processus de Markov), la Loss du réseau et les gradients.

Pour réduire la corrélation entre chaque donnée, on peut juste prendre un buffer (une mémoire) qui va stocker les tuples (S, A, P, R) et on pioche des mini-batch dans ces données pour que le réseau puisse généraliser.

Target Network

Si on utilise le même réseau pour prédire la valeur actuelle $Q(s, a)$ et calculer les estimations futures $Q(s', a')$, quand on va mettre à jour les poids du réseau, on change aussi son estimation sur s' , donc on peut pas converger.

On utilise un réseau pour l'entraînement de $Q(s, a)$ et un réseau qui calcule les $Q(s', a')$.

Tous les 1000 pas de temps par exemple, on synchronise les deux réseaux.

II. Deuxième proposition

1. Titre du projet: agent IDS – Intrusion Detection System.

2. Problématique: Ici, le but est de concevoir un pare-feu capable de filtrer le trafic réseau de manière autonome. Contrairement à un pare-feu classique basé sur des règles comme "bloquer le port 80" et les iptables, notre agent doit apprendre par l'expérience à détecter des comportements anormaux.

Composants du système:

- On a l'agent (le pare-feu) qui analyse les statistiques des flux et décide des actions à prendre.
- Le flux de données (environnement): la séquence des paquets arrivant sur l'interface réseau.
- Un capteur qui extrait les caractéristiques du trafic (IP, ports, tailles, flags)
- Le moteur de filtrage qui applique la décision (Drop, Pass, ...)

Performance et objectifs:

- La sécurité: bloquer 100% des attaques (éviter les faux négatifs)
- La disponibilité: ne pas bloquer le trafic légitime (éviter les faux positifs)

3. Définition du MDP

- Etats (S): un vecteur représentant les caractéristiques des N derniers paquets avec des infos réseaux classiques (à préciser). Par exemple:

[Taille moyenne, Fréquence d'arrivée, Nombre d'IP sources distinctes, FLAGS, ...]

- Actions (A): un ensemble {AUTORISER, BLOQUER, LIMITER LE DEBIT, ANALYSE PROFONDE}. L'idée est d'avoir une main sur la QoS notamment le débit afin d'éviter des attaques type DDoS ou Brute-Force. La DPI sur du CPU dans un vrai pare-feu peut être lente. Du coup ici ça peut être un compromis intéressant pour être certain à 100% tout en gardant des choix rapides en moyenne.

- Transitions (fonction continue P): elles impactent la fenêtre d'observation. Par exemple, bloquer une IP modifie le futur trafic que l'agent verra passer.

- Récompenses (R): Exemple

+1 pour chaque paquet légitime autorisé

+10 pour une attaque bloquée

-100 si une attaque passe

-20 si un paquet légitime ou utilisateur légitime est bloqué (on casse aussi le service)

- Paramètres: on prend un learning rate assez faible pour pas que le réseau oublie des attaques dès qu'une nouvelle arrive.

Aussi un gamma assez élevé pour miser sur le long terme. On commence avec un epsilon élevé pour l'exploration et on le diminue jusqu'à 0.1 par exemple.

4. Implémentation

On choisit de coder en C car on aime bien le bas niveau.

Pour la simulation, le simulateur va en fait lire un fichier de logs (un dataset). Chaque ligne est un état. Le code simule l'arrivée des paquets. L'agent reçoit l'état et choisit l'action. On va le faire en ligne de commande (dans la console) et ensuite exporter les résultats pour les visualiser avec Python.

On utilise du Deep Q-Learning (pour les mêmes raisons qu'expliqué ci-dessus).

Détails algorithmiques:

On va utiliser la MSE comme Loss entre l'équation de Bellman $r + \text{Gamma} * \max Q(s', a')$ et la prédiction actuelle $Q(s, a)$.

$$L = [r + \text{Gamma} * \max Q(s', a') - Q(s, a)]^2.$$

Attention, les Q-values peuvent avoir une valeur réelle quelconque. On ne peut pas choisir de Sigmoidé en sortie par exemple. Il faut une activation linéaire.

On doit combiner nos méthodes avec l'Experience Replay et le Target Network comme pour le premier sujet proposé. En fait, ce sont des optimisations souvent nécessaires qui sont la conséquence de l'utilisation d'un réseau de neurones.

Copié-collé de ce qui est mis ci-dessus:

Experience Replay

Pour traiter des espaces continus, il faut implémenter de l'*Experience Replay*. Le principe de base est que d'un pas de temps à l'autre, les données sont très similaires donc le réseau va faire de l'overfitting sur sa position actuelle et prendre les mauvaises décisions.

Pour réduire la corrélation entre chaque donnée, on peut juste prendre un buffer (une mémoire) qui va stocker les tuples (S, A, P, R) et on pioche des mini-batch dans ces données pour que le réseau puisse généraliser.

Target Network

Si on utilise le même réseau pour prédire la valeur actuelle $Q(s, a)$ et calculer les estimations futures $Q(s', a')$, quand on va mettre à jour les poids du réseau, on change aussi son estimation sur s' , donc on peut pas converger.

On utilise un réseau pour l'entraînement de $Q(s, a)$ et un réseau qui calcule les $Q(s', a')$.

Tous les 1000 pas de temps par exemple, on synchronise les deux réseaux.